

## BFS

### Versión inicial:

```
def breadthFirstSearch(problem: SearchProblem):  
    """  
    Search the shallowest nodes in the search tree first.  
    """  
    start = problem.getStartState()  
    frontier = utils.Queue()  
    #agregar el estado inicial a la frontera. La lista de acciones es vacia  
    frontier.push((start, []))  
    while not frontier.isEmpty():  
        state, actions = frontier.pop()  
        if problem.isGoalState(state):  
            return actions  
        for successor, action, _ in problem.getSuccessors(state):  
            #Se agregan los sucesores a la frontera con las acciones  
            correspondientes  
            frontier.push((successor, actions + [action]))  
    return []
```

### Prompt usado:

Revisa el breadthFirstSearch

### Sugerencia relevante de IA:

La implementación usa una cola, pero no registra estados visitados. En mapas con ciclos puede volver a encolar los mismos estados indefinidamente. Se recomienda usar un conjunto visited y marcar los sucesores al encolarlos.

### Cambio aceptado:

```
def breadthFirstSearch(problem: SearchProblem):  
    """  
    Search the shallowest nodes in the search tree first.  
    """  
    start = problem.getStartState()  
    frontier = utils.Queue()  
    frontier.push((start, []))  
    visited = {start}  
  
    while not frontier.isEmpty():  
        state, actions = frontier.pop()  
        if problem.isGoalState(state):  
            return actions  
        for successor, action, _ in problem.getSuccessors(state):  
            if successor not in visited:  
                visited.add(successor)  
                frontier.push((successor, actions + [action]))  
    return []
```

### Verificación:

Se probó ejecutando python menu.py, seleccionando el algoritmo BFS y el mapa correspondiente. Al ejecutarlo, la terminal mostró el resultado de la búsqueda, incluyendo el costo total del camino encontrado y el número de nodos expandidos, confirmando que la implementación funciona correctamente.

## systemRepairHeuristic

### Versión inicial (sin cache):

```
def systemRepairHeuristic(state, problem):
    position, hasKit, pendingSystems = state

    if hasKit:
        points = [position] + list(pendingSystems) + [problem.controlPosition]
        return MST(points)

    else:
        distancia_kit = distancia(position, problem.kitPosition)
        points = [problem.kitPosition] + list(pendingSystems) +
        [problem.controlPosition]
        return distancia_kit + MST(points)
```

### Prompts usados:

no entiendo para el systemrepairheuristic como funciona state / me puedes ayudar a revisar esa implementacion del systemrepairheuristic? / Que es best\_point / Como funciona problem.heuristicInfo? / Asi queda mejor? / Y ahora como esta? / y este es mas rapido?

### Respuestas / sugerencias relevantes:

En SystemRepairProblem, state tiene la forma (position, hasKit, pendingSystems). position indica la posición actual del robot, hasKit indica si ya recogió el kit K, y pendingSystems contiene las posiciones T que aún faltan por reparar. Ejemplo: ((2, 10), False, ((5, 3), (9, 4), (12, 6))) significa que el robot está en (2, 10), todavía no tiene el kit y faltan por reparar los sistemas en (5, 3), (9, 4) y (12, 6).

Sobre la heurística, se explicó que debe retornar un número que estime el costo restante, y se sugirió usar distancia Manhattan y revisar la construcción de la lista de puntos para el MST. También se sugirió guardar el punto más cercano en una variable durante el cálculo del MST.

Se explicó que problem.heuristicInfo es un diccionario asociado al problema, útil para guardar cálculos costosos de la heurística (como el MST) y reutilizarlos cuando se repite el mismo conjunto de puntos. Se recomendó guardar el resultado de MST(points) usando una clave basada en los puntos involucrados.

### Cambios aceptados:

Se cambió la construcción de la lista de puntos, de:

```
points = [position, list(pendingSystems), problem.controlPosition]
```

a:

```
points = [position] + list(pendingSystems) + [problem.controlPosition]
```

También se aceptó la idea de guardar el punto más cercano en una variable `best` durante el cálculo del MST.

Además, se agregó una clave para identificar cada cálculo de MST:

```
key = ("mst", tuple(sorted(points)))
```

y se usó `problem.heuristicInfo` para evitar recalcular:

```
if key not in problem.heuristicInfo:
    problem.heuristicInfo[key] = MST(points)

return problem.heuristicInfo[key]
```

### **Versión final (con cache):**

```
def systemRepairHeuristic(state, problem):
    position, hasKit, pendingSystems = state

    if hasKit:
        points = [position] + list(pendingSystems) + [problem.controlPosition]
        key = ("mst", tuple(sorted(points)))

        if key not in problem.heuristicInfo:
            problem.heuristicInfo[key] = MST(points)

        return problem.heuristicInfo[key]

    else:
        distancia_kit = distancia(position, problem.kitPosition)
        points = [problem.kitPosition] + list(pendingSystems) +
[problem.controlPosition]
        key = ("mst", tuple(sorted(points)))

        if key not in problem.heuristicInfo:
            problem.heuristicInfo[key] = MST(points)

        return distancia_kit + problem.heuristicInfo[key]
```

### **Cambio corregido con apoyo de IA:**

En una versión intermedia, la clave se había puesto antes de definir `points`:

```
key = tuple(sorted(points))
```

La IA señaló que eso produciría un error porque `points` todavía no existía. Se corrigió moviendo la creación de `key` después de construir la lista `points`.

### **Cambios rechazados:**

No se hicieron cambios adicionales a la lógica del MST. El cache solo evita recomputar valores ya calculados; no modifica la estimación de la heurística.

### **Verificación:**

Se probó ejecutando `python menu.py`, seleccionando la heurística `systemRepairHeuristic` junto con el mapa correspondiente. Al ejecutarlo, la terminal mostró el resultado de la búsqueda, incluyendo el

costo total y el número de nodos expandidos, confirmando que la heurística con cache funciona correctamente.